

Neurolabs iOS SDK Integration Guide

Technical Integration Specification

Document ID: NLB-IOS-INTEGRATION-GUIDE

Version: v1.6.6

Date: 2026-07-22

Status: Partner integration reference

Owner: Neurolabs

1. Scope

This guide is for partner iOS apps integrating 'NeurolabsSDK' through Swift Package Manager.

2. Release History

No public API contract has been broken since v1.1.9. New options have been added with backwards-compatible defaults. Recommended defaults: strict shelf guidance with overlays disabled, 'liveQualityEnabled: true', 'autoCloseAfterCapture: true' for single-capture parity flows. 'NLCameraConfiguration.captureRange(...)' is the preferred helper for custom-camera flows that need a minimum capture count and a higher maximum cap. 'showsCapturedCountLabel' hides the 'N captured' chip in custom camera and multi-capture flow UIs.

v1.3.2

- Mixpanel ingestion routes to the EU region via

'NLANalyticsContext.resolveMixpanelEndpoint(fallback: .mixpanelEndpointEU)' at SDK construction; the 'NLMixpanelAnalyticsTracker' default itself stays on the standard US endpoint so partners reusing the class for their own US-region projects are unaffected. Endpoint can be overridden via the 'NLMixpanelEndpoint' Info.plist key or the 'NL_MIXPANEL_ENDPOINT' environment variable (env wins over plist).

- Mixpanel 'time' field is now serialized as a JSON number (epoch

seconds). Previously 'time' was sent as a string, which Mixpanel rejected with '{"status":0,"error":"time field must be a number"}', silently dropping every event.

- 'NLCustomCameraView.saveAllCaptures' keeps the Done button responsive:

the synchronous 'FileManager.removeItem' loop for staged-capture files moved into a 'Task.detached(priority: .utility)' block, and the Done label swaps to an inline 'ProgressView' while 'isPersistingCaptures == true'. Opacity remains at 1 during persist so the indicator stays visible.

- New 'NLOperationsCatalogClient' (in 'ProductAuditKit') for the operations product catalog, with 'BarcodeLookupIdentity' normalising UPC-E/EAN-8/UPC-A/EAN-13 to GTIN-14 end-to-end.

'NLOperationsCatalogError.invalidBarcode' / '.invalidSearchQuery' / '.malformedResponse' are thrown rather than crashing the host with 'precondition'.

- 'NLBDemoApp' gains an in-app settings sheet on 'CustomCameraDemoView'

(gear icon, top-right). Sliders / steppers / toggles for every tunable knob, bindings wired to existing '@AppStorage' properties so saved values survive across launches.

- Mixpanel event identity hardened: per-install distinct id persisted in 'UserDefaults', '\$insert_id' (UUID4) on every event for server-side dedup, response body surfaced when 'debugLogging' is true.

v1.3.1

- Restrict barcode symbologies to retail formats (EAN-13, EAN-8, UPC-A, UPC-E) so the scanner stops emitting non-product codes during capture.

v1.3.0

- Angle-off warning threshold relaxed to 12 for less aggressive blocking during normal in-store motion.
- Multi-capture preview deletion: persisted thumbnails are removed when the matching capture is dropped.
- AVFoundation preview recovery hardened against backgrounding / interruption.

v1.2.8

- 'ProductAuditKit' test targets added (no public API change).

v1.2.6 / v1.2.7

- Product Audit (onboarding) MVP: serialise 'AVAssetWriter' finalize and 'OCRSmartRotationProvider' work via in-flight 'Task', off-actor keyframe JPEG writes, completedStepIDs reflect steps that actually ran, step transitions serialised via pending-transition queue.

v1.2.5

- New 'showsSequenceCounterLabel' option in the native capture API for toggling the on-screen sequence counter.
- Persistent custom-camera tuning in the demo app (precursor to the v1.3.2 settings sheet) - preferences survive across runs via 'UserDefaults'.
- AVFoundation preview recovery improved on capture failure.
- Strict-rejection preview freeze regression fixed.

v1.2.4

- Release build / Swift 6 archive blockers resolved.

v1.2.3

- 'NLCameraConfiguration.captureRange(...)' helper for bounded multi-capture flows.
- 0.5/1 lens switcher built on AVFoundation, default camera mode starts on the ultra-wide lens when available. Tap-to-focus, lens switch fallback covered by unit tests.
- Custom-camera startup, lens switching, and guidance responsiveness improvements; camera preview shown during warmup, capture button gated until ready.
- Default task routing config uses 'taskUUID'.

- Recommended custom-camera parity flow remains strict shelf guidance with overlays disabled, 'liveQualityEnabled: true', 'autoCloseAfterCapture: true' with a single required capture.

v1.2.2

- Analytics tracking + opt-out, partner/device context, Mixpanel defaults, Sentry integration with privacy-safe defaults.

v1.2.1

- Initial lens switcher, memory fixes, bottom-bar centering and UI polish.

v1.2.0

- Docs aligned to v1.1.9 requirements baseline; no SDK code change.

v1.1.9 (baseline)

- Init and per-session routing use 'taskUUID'.
- Native detector/model warmup enforced in 'init' and guarded in 'openCamera'.
- 'autoCloseAfterCapture: true' keeps preview enabled and closes after Save confirmation.

3. Requirements

- iOS 17.0+ (SDK package declares '.iOS(.v17)')
- Xcode 16.4+ (release toolchain baseline)
- Swift 5.9+ (Swift 6 toolchains supported)

4. Install (SPM)

Add package dependency (mobile-dist or direct distribution manifest) and link 'NeurolabsSDK'.

5. SDK Initialization + Warmup

Uploads use the **operations platform** with a **single key**: the 'apiKey' passed to 'init' seeds the operations credential automatically, and every capture routes through the resumable operations mission flow ('init-resumable chunked PUT complete' per image, 'submit' per session) against 'api.operations.neurolabs.ai'. No base URL or separate credential call is needed; use 'setOperationsApiKey(_:baseURL:)' only to rotate the key or to point at a non-production host (staging/dev).

```

import NeurolabsSDK

final class CaptureCoordinator {
    let sdk = NeurolabsSDKCore(configuration: .init(
        apiKey: "<OPERATIONS_API_KEY>",
        debugLogging: true,
        taskUUID: "<DEFAULT_TASK_UUID>" // optional IR-task override (never a mission id)
    ))

    func prepare() {
        // SDK starts image pipeline preparation during init.
        // Optional: observe loading progress through delegate.
    }
}

```

6. Queue Management

```

Task {
    await sdk.setAutoSyncEnabled(true)
    await sdk.setWifiOnlyUploadsEnabled(false)
    await sdk.setDeletePhotosOnUploadSuccess(true)
    await sdk.setMaxQueueSize(200)
    await sdk.setUploadRetryCount(5)

    let status = await sdk.getQueueStatus()
    print("pending=\(status.pendingCount) failed=\(status.failedCount)")

    await sdk.flushQueue()
    await sdk.retryFailedUploads()
}

```

7. Open Custom Camera

```
import UIKit
import NeurolabsSDK

var capture = NLCaptureConfiguration()
capture.confidenceThreshold = 0.25
capture.iouThreshold = 0.45
capture.maxCaptures = 1
capture.guidanceMode = .strict
capture.showDetectionOverlays = false
capture.showCapturedRegions = false

let config = NLCameraConfiguration.captureRange(
    capture,
    minimumCaptures: 1,
    maximumCaptures: 5,
    cameraMode: .default,
    showCapturePreview: true,
    showPreviewInStrictMode: true,
    showsCapturedCountLabel: false,
    enableValidation: true,
    showAlignmentGuidance: true,
    enableARDetections: false,
    liveQualityEnabled: true,
    liveQualityFPS: 6
)

let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        // Custom post-processing hook for each accepted capture
        // return .performDefault -> SDK also enqueues upload
        // return .skipDefault -> app takes full ownership
        return .performDefault
    },
    onSaveAll: { captures in
        // Batch hook (multi-capture flow)
        return .performDefault
    }
)

sdk.openCustomCameraUI(
    configuration: config,
    handlers: handlers,
    onClose: {
        print("Camera closed")
    }
)
```

Recommended capture payload notes:

- 'liveQualityEnabled: true' is the key for pill/rotation/warning/error guidance behavior in the custom camera.
- Keep shelf capture mode with strict guidance and overlays disabled for the custom-camera guidance UI path.
- Use 'NLCameraConfiguration.captureRange(...)' to express a minimum capture requirement plus a larger maximum cap.
- Use 'showsCapturedCountLabel = false' to hide the count chip when you do not want 'N captured' shown.

- 'guidanceMode: .guidance' should only be used as a temporary fallback if a partner explicitly wants looser guidance than the parity path.

8. WebView Bridge Example

The same capture-range options are available through the native bridge models:

```
let config = NLNativeCaptureConfig(
    guidanceMode: .strict,
    showCloseButton: false,
    maxCaptures: 5,
    allowManualFinish: true,
    minCapturesBeforeDone: 1,
    showsCapturedCountLabel: false,
    cameraMode: .default
)

neurolabsSDK.openNativeCaptureUI(config: config, sessionId: sessionId)
```

Use 'NLNativeCaptureOptions' with the same field names when the bridge payload is built from JS or other app code.

9. Post-Processing Hooks

- 'onSave' receives each approved capture.
- 'onSaveAll' receives final batch in multi-capture mode.
- 'onUploadRequested' exists but default queueing is done in 'onSave' / 'onSaveAll' path.

```
let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        MyPostProcessor.shared.enqueue(capture)
        return .skipDefault
    }
)
```

9. Delegate/Event Callbacks

```
final class SDKDelegate: NeurolabsSDKDelegate {
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didProduceCaptureResult result: NLNativeCaptureResult, for captureId: String, sessionId: String) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didEncounterError error: NSError, sessionId: String?, messageId: String?) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didChangeQueueStatus status: NLQueueStatus) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didUpdateLoadingProgress progress: NLLoadingProgress) {}
}

sdk.delegate = SDKDelegate()
```

10. Notes

- Per-session routing is supported via native capture config task UUID overrides.
- Ensure 'NSCameraUsageDescription' is present in app 'Info.plist'.